

Scenario:

The e-commerce company wants to maintain information about products and their categories. Each product belongs to a specific category, and the system should manage product details such as name, price, and stock availability.

1. Design Product and Category Tables

Category Table:

```
SQL> CREATE TABLE CATEGORY(CATEGORY_ID VARCHAR(7),CATEGORY_NAME VARCHAR(20),  
DESCRIPTION VARCHAR(50));
```

Product Table:

```
SQL> CREATE TABLE PRODUCT(PRODUCT_ID VARCHAR(7),PRODUCT_NAME VARCHAR(20),CATEGORY_ID VARCHAR(7),PRICE NUMBER(9,2),STOCK_QUANTITY NUMBER);  
  
Table created.
```

The two tables for Category and Product have been created.

Now, we have to find the relationship between the tables and apply constraints to the tables.

2. Define Relationships and Constraints

Relationships:

One Category Can have many Products.

The relationship between Category and Product is One to many (1-> *).

We found that, what is their relationship. Now we have to discuss that which is the key that is going to make this relationship.

Before that, we need to understand about KEY Constraints.

PRIMARY KEY:

The Column(s) or Field(s) that uniquely identifies a row / tuple

In these two tables, which field can be a Primary Key?

In Category Table, Category_ID is a unique Value that helps to identify each category. Likewise, in Product Table Product_ID is a unique Column.

FOREIGN KEY:

The Key which links two tables.

Therefore, Category_ID acts as Foreign Key in the Product Table.

We can explore some other constraints like NOT NULL, CHECK and UNIQUE

NOT NULL:

Ensures that the Column cannot have a empty (null) value.

For Example, a Product name or Category name cannot be empty. All category or product must have a name.

CHECK:

Validates the values in a column satisfies the condition.

For Example, To check the Price of a Product must always be a positive number.

UNIQUE:

Ensures that there is no repeated value for a column.

For Example, If we mention UNIQUE Constraint to Category Name, no two category can have same name.

Defining Relationship and Creating Constraints for Product and Category:

Primary Key:

```
SQL> ALTER TABLE CATEGORY ADD CONSTRAINT PK_CATEGORY PRIMARY KEY (CATEGORY_ID);  
Table altered.
```

```
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

Creating Relationship using FOREIGN KEY:

```
Connected.  
SQL> ALTER TABLE Product ADD CONSTRAINT Fk_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);  
Table altered.
```

Adding Constraints:

NOT NULL:

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;
Table altered.

SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;
Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_PRICE CHECK (Price > 0);
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);
Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category UNIQUE (Category_Name);
Table altered.
```

3. Insert Values:

```
SQL> INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description');
Enter value for category_id: 11
Enter value for category_name: Electronics
Enter value for description: Electronic devices and accessories
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('11','Electronics','Electronic devices and accessories')
1 row created.

SQL> /
Enter value for category_id: 12
Enter value for category_name: Clothing
Enter value for description: Clothes and fashion wear
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('12','Clothing','Clothes and fashion wear')
1 row created.

SQL> /
Enter value for category_id: 13
Enter value for category_name: Books
Enter value for description: Books,novels and study materials
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('13','Books','Books,novels and study materials')
1 row created.

SQL> /
Enter value for category_id: 14
Enter value for category_name: Sports
Enter value for description: Sports equipment and accessories
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('14','Sports','Sports equipment and accessories')
1 row created.

SQL> /
Enter value for category_id: 15
Enter value for category_name: Furniture
Enter value for description: Home and office furniture
```

Product Table:

```
SQL> INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity);
Enter value for product_id: P11
Enter value for product_name: Bluetooth Speaker
Enter value for category_id: 01
Enter value for price: 2500
Enter value for stock_quantity: 20
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P11','Bluetooth Speaker','01',2500,20)

1 row created.

SQL> /
Enter value for product_id: P12
Enter value for product_name: Cotton Shirt
Enter value for category_id: 02
Enter value for price: 899
Enter value for stock_quantity: 25
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P12','Cotton Shirt','02',899,25)

1 row created.
```

```
SQL> /
Enter value for product_id: P13
Enter value for product_name: English Grammar Book
Enter value for category_id: 03
Enter value for price: 450
Enter value for stock_quantity: 15
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P13','English Grammar Book','03',450,15)

1 row created.

SQL> /
Enter value for product_id: P14
Enter value for product_name: Cricket Bat
Enter value for category_id: 04
Enter value for price: 1800
Enter value for stock_quantity: 10
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P14','Cricket Bat','04',1800,10)

1 row created.

SQL> /
Enter value for product_id: P15
Enter value for product_name: Study Table
Enter value for category_id: 05
Enter value for price: 4500
Enter value for stock_quantity: 8
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P15','Study Table','05',4500,8)

1 row created.
```

Like this insert few more rows.

After Insertion Category Table:

```
SQL> SELECT * FROM CATEGORY;
```

CATEGOR	CATEGORY_NAME	DESCRIPTION
01	Electronics	Electronic devices and accessories
02	Clothing	Clothes and fashion wear
03	Books	Books and study materials
04	Sports	Sports equipment and accessories
05	Furniture	Home and office furniture

Product Table:

```
SQL> SELECT * FROM Product;
```

PRODUCT	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
P11	Bluetooth Speaker	01	2500	20
P12	Cotton Shirt	02	899	25
P13	English Grammar Book	03	450	15
P14	Cricket Bat	04	1800	10
P15	Study Table	05	4500	8

CRUD Operations:

Insert Record:

```
SQL> INSERT INTO Product (PRODUCT_ID,PRODUCT_NAME,PRICE,STOCK_QUANTITY) VALUES('P06','Non-Stick Pan','1200','18');  
1 row created.
```

Update Price:

```
SQL> UPDATE Product SET Price = 550 WHERE Product_Name = 'English Grammar Book';  
1 row updated.
```

Delete a Product:

```
SQL> DELETE FROM Product WHERE Product_Name = 'Study Table';  
1 row deleted.
```

5. Generate Category-wise Product Reports

Write SQL queries to:

- Display all products under each category.

```
SQL> SELECT c.Category_Name,p.Product_Name,p.Price,p.Stock_Quantity FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID;
```

CATEGORY_NAM	PRODUCT_NAME	PRICE	STOCK_QUANTITY
Electronics	Bluetooth Speaker	2500	20
Clothing	Cotton Shirt	899	25
Books	English Grammar Book	550	15
Sports	Cricket Bat	1800	10

- Count number of products in each category.

```
SQL> SELECT c.Category_Name,COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAM	PRODUCT_COUNT
Books	1
Electronics	1
Clothing	1
Sports	1

- Find highest-priced product in each category.

```
SQL> SELECT c.Category_Name,p.Product_Name,p.Price AS Highest_price FROM Product p JOIN Category c ON p.Category_ID=c.Category_ID WHERE p.Price=(SELECT MAX(p2.Price) FROM Product p2 WHERE p2.Category_ID=p.Category_ID);
```

CATEGORY_NAM	PRODUCT_NAME	HIGHEST_PRICE
Electronics	Bluetooth Speaker	2500
Clothing	Cotton Shirt	899
Books	English Grammar Book	550
Sports	Cricket Bat	1800

- Display categories having more than 2 products.

```
SQL> SELECT c.Category_Name FROM Product p JOIN Category c ON p.Category_ID=
c.Category_ID GROUP BY c.Category_Name HAVING COUNT(p.Product_ID)>2;

no rows selected
```

- Find average product price category-wise.

```
SQL> SELECT c.Category_Name,AVG(p.Price) AS Avg_Price FROM Product p JOIN Ca
tegory c ON p.Category_ID=c.Category_ID GROUP BY c.Category_Name;

CATEGORY_NAM  AVG_PRICE
-----
Books          550
Electronics    2500
Clothing       899
Sports         1800
```